4. Permissions Basics: rwx, chmod, chown

04. Permissions Basics: rwx, chmod, chown

Phase: Phase 2 — Permissions and identity
Estimated time: 1–2 days
Slug: `04-permissions-basics`

Learning objectives

- Read `ls -l` permission bits
- Change mode with chmod (symbolic and octal)
- Change owner/group with chown/chgrp
- Predict access for user/group/other

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

Permissions are the main safety net on a multi-user system. They decide who can read your SSH keys, who can run a script and who can change a config file. Reading `-rwxr-x---` fluently and choosing correct modes with `chmod` and `chown` prevents both “Permission denied” headaches and dangerous world-writable files.

Core concepts

Every file has an owner (user), a group and three permission sets: user (u), group (g) and others (o). Each set has read (r), write (w) and execute (x). For directories, r means list names, w means create/delete/rename entries, x means enter and access entries (traverse).

```
-  rwx  r-x  r--   alice  devs  report.sh
|   |    |    |
|   |    |    +-- others: read only       (4)
|   |    +------- group:  read + execute  (5 = 4+1)
|   +------------ user:   read/write/exec (7 = 4+2+1)
+---------------- type: - file, d directory, l link
mode octal: 754
```

Octal	Symbolic	Meaning
7	rwx	all
6	rw-	read, write
5	r-x	read, execute
4	r–	read only
0	—	none

Each digit is the sum r=4, w=2, x=1. Common modes: `644` (`rw-r--r--`) for ordinary files, `755` (`rwxr-xr-x`) for programs and directories, `600` (`rw-------`) for private keys, `700` for private directories.

`chmod` accepts octal (`chmod 640 f`) or symbolic (`chmod u+x f`, `chmod go-w f`, `chmod a=r f`). `chown user:group file` changes ownership (root only for changing the owner); `chgrp` changes only the group; `-R` recurses. New files get mode `0666 & ~umask` (files) or `0777 & ~umask` (directories); with umask `022` that is 644 and 755.

Worked examples

Read a listing:

```
ls -l deploy.sh
# sample output: -rwxr-x--- 1 alice devs 214 Oct 21 10:00 deploy.sh
```

Alice can do everything, group `devs` can read and run, others get nothing.

Change modes:

```
chmod 640 deploy.sh
ls -l deploy.sh
# sample output: -rw-r----- 1 alice devs 214 Oct 21 10:00 deploy.sh
chmod u+x,g-r deploy.sh
ls -l deploy.sh
# sample output: -rwx------ 1 alice devs 214 Oct 21 10:00 deploy.sh
stat -c '%a %A %U:%G' deploy.sh
# sample output: 700 -rwx------ alice:devs
```

Change ownership (requires root):

```
sudo chown bob:devs deploy.sh
ls -l deploy.sh
# sample output: -rwx------ 1 bob devs 214 Oct 21 10:00 deploy.sh
```

Directory execute bit:

```
mkdir d && touch d/f && chmod 644 d
ls d
# sample output: f
```

```
cat d/f
# sample output: cat: d/f: Permission denied
```

Listing names needs r, but reaching `d/f` needs x on `d`.

Umask:

```
umask
# sample output: 0022
touch n && mkdir m && stat -c '%a %n' n m
# sample output: 644 n
# sample output: 755 m
```

Mini-lab

1. `cd /tmp && mkdir perm && cd perm && touch f && ls -l f`.
2. Convert the mode you see to octal by hand, then confirm with `stat -c %a f`.
3. `chmod 000 f; cat f` fails (as root it may not; use a normal user or `su`).
4. `chmod u+r f; cat f` works; `chmod 755 f; ls -l f`.
5. Create `s.sh` containing `echo ok`; run `./s.sh` (denied), `chmod +x s.sh`, run again.
6. Set `umask 077`, create a file and a directory, check their modes, then `umask 022`.
7. `mkdir secret && chmod 700 secret`; test access as another user with `sudo -u nobody ls secret`.
8. Convert 640, 755, 600 and 711 to symbolic form on paper, then verify using `chmod` and `ls -l`.

Decision guide / failure modes

Use octal when you know the full target mode; use symbolic (`u+x`) to change one bit without touching others. Give files 644, scripts 755, private keys 600, shared project directories 770 or 2770 with a group (module 6). Never use 777 as a fix.

Common failures:

- `chmod -R 755 .` makes every data file executable; use `find -type d` and `find -type f` separately.
- SSH refuses a key with “UNPROTECTED PRIVATE KEY FILE”: set `chmod 600 ~/.ssh/id_*` and `700 ~/.ssh`.
- Script gives “Permission denied” because it lacks `x` or sits on a `noexec` mount.
- Forgetting directory `x`, so files inside can be listed but not opened.
- `chown -R` with a wrong path or a trailing `/..`; double-check before running as root.

Common mistakes

- chmod 777 on everything “to make it work”
- Forgetting directories need `x` to be entered
- Running as root to avoid learning permissions

Exercises

Work in `exercises/04-permissions-basics/`. Read that folder's `README.md` fully before coding.

```
cd exercises/04-permissions-basics
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement octal_to_human(mode). The input is a three-digit octal permission string such as "755" or "640" (digits 0-7). Return the nine-character symbolic form without the file-type character, like "rwxr-xr-x". Each digit expands to three characters: 4 gives r, 2 gives w, 1 gives x, and missing bits are "-". Digits are in order user, group, others. Do not use the filesystem. You may assume the input is valid three digits; for anything else return an empty string. This is the same mapping chmod and ls -l use.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/04-permissions-basics/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. What does 644 mean?
2. Why does a directory need execute?
3. When is 777 inappropriate?

Further reading

- https://man7.org/linux/man-pages/man1/chmod.1.html